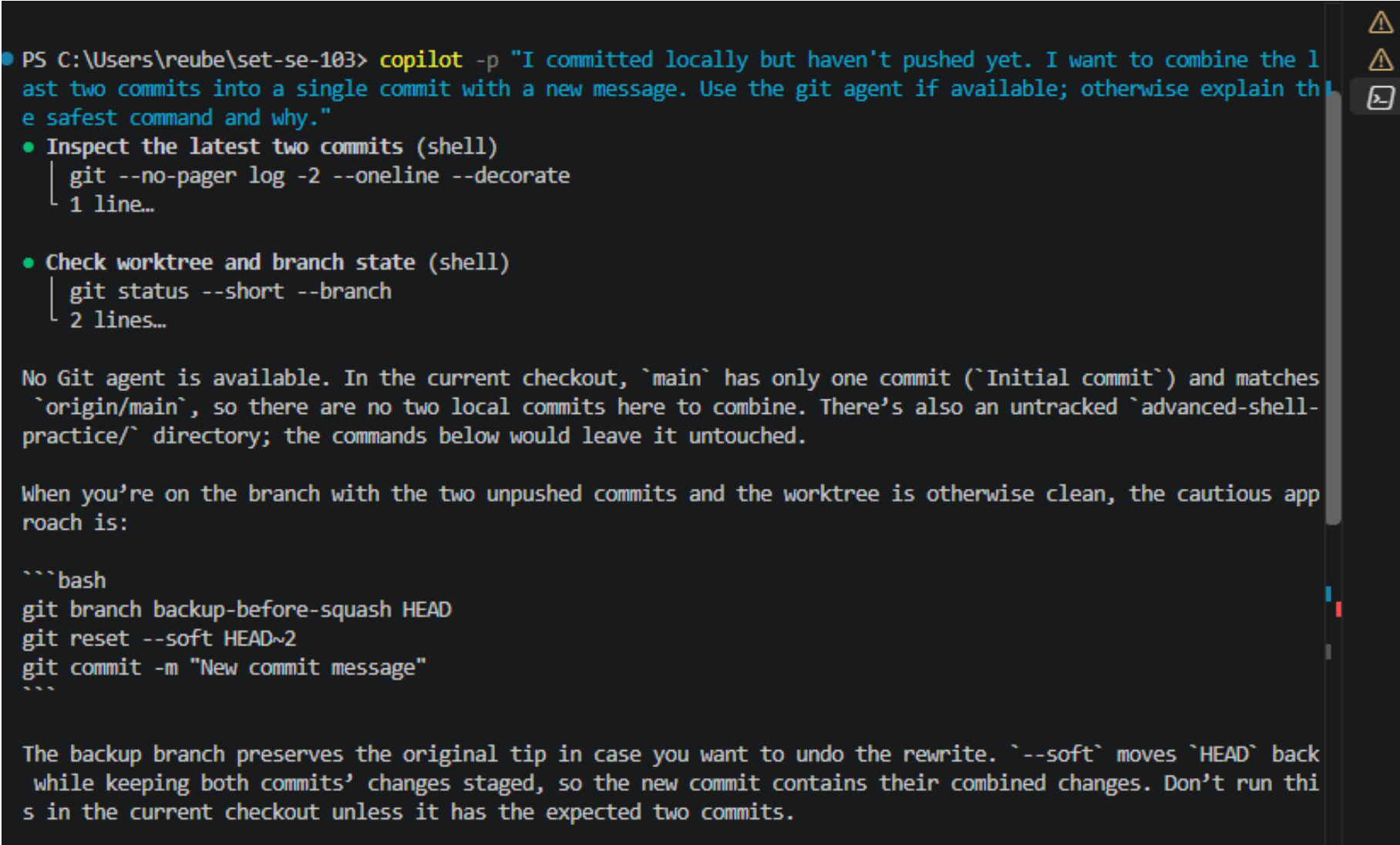


Reuben_AI: Mastering the Shell with GitHub Copilot CLI

Copilot CLI terminal screenshot

A terminal window with a dark background. The prompt is 'PS C:\Users\reube\set-se-103>'. The user enters 'copilot -p "I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message. Use the git agent if available; otherwise explain the safest command and why."' followed by a blue cursor. The response lists two suggestions: 'Inspect the latest two commits (shell)' with command 'git --no-pager log -2 --oneline --decorate' and 'Check worktree and branch state (shell)' with command 'git status --short --branch'. Below these, a message explains that no Git agent is available and provides a cautious approach using 'git branch backup-before-squash HEAD', 'git reset --soft HEAD~2', and 'git commit -m "New commit message"'. The final paragraph explains the purpose of the backup branch and the --soft flag.

```
PS C:\Users\reube\set-se-103> copilot -p "I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message. Use the git agent if available; otherwise explain the safest command and why."
```

- Inspect the latest two commits (shell)
 - | git --no-pager log -2 --oneline --decorate
 - | 1 line...
- Check worktree and branch state (shell)
 - | git status --short --branch
 - | 2 lines...

No Git agent is available. In the current checkout, `main` has only one commit (`Initial commit`) and matches `origin/main`, so there are no two local commits here to combine. There's also an untracked `advanced-shell-practice/` directory; the commands below would leave it untouched.

When you're on the branch with the two unpushed commits and the worktree is otherwise clean, the cautious approach is:

```
```bash
git branch backup-before-squash HEAD
git reset --soft HEAD~2
git commit -m "New commit message"
```
```

The backup branch preserves the original tip in case you want to undo the rewrite. `--soft` moves `HEAD` back while keeping both commits' changes staged, so the new commit contains their combined changes. Don't run this in the current checkout unless it has the expected two commits.

The screenshot shows the complete Copilot CLI prompt and response, including the suggested commands and safety explanation.

Git command and CLI follow-up

CLI prompt: "I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message. Use the git agent if available; otherwise explain the safest command and why."

This prompt gives the local/unpushed context, states the desired history change, names the preferred git agent, and requests a safety explanation. That specificity asks the CLI to address the real Git workflow rather than return a generic command.

The terminal CLI reported that no git-named agent was available, so it adapted by using shell checks. It inspected the latest commits and branch status, found that main had only one commit and matched origin/main, and correctly warned that there were no two local commits to combine. It also noted the untracked assignment directory and said the proposed commands would leave it untouched.

Command for a branch with at least two local, unpushed commits

```
git branch backup-before-squash HEAD
git reset --soft HEAD~2
git commit -m "New commit message"
```

The backup branch preserves the original tip. The soft reset then moves the current branch back two commits while keeping their combined changes staged, and the new commit records those changes under one message. These commands were not run: the current checkout did not meet the two-commit condition. A clean, verified worktree and the intended branch should be confirmed before applying them.

Accuracy ranking and justification

Rating: 8/10. The CLI made this task more efficient by checking the actual branch and commit history before recommending a history rewrite; that prevented me from applying HEAD~2 to a checkout with only one commit. Its suggested sequence is appropriate for two local, unpushed commits, and the backup branch adds a recovery point. I did not give it 10/10 because the requested git agent was unavailable and the workflow could not be demonstrated on the intended two-commit history. If commits have already been pushed, rewriting shared history can disrupt collaborators and any force-push must be coordinated.

Comparison with git rebase -i

An interactive rebase, such as `git rebase -i HEAD~2`, lets me choose squash or fixup and edit the combined message in the sequence editor. It is preferable when I need to reorder, drop, or selectively edit commits. The soft-reset approach is more direct when I simply want to combine exactly the last two commits under one new message.